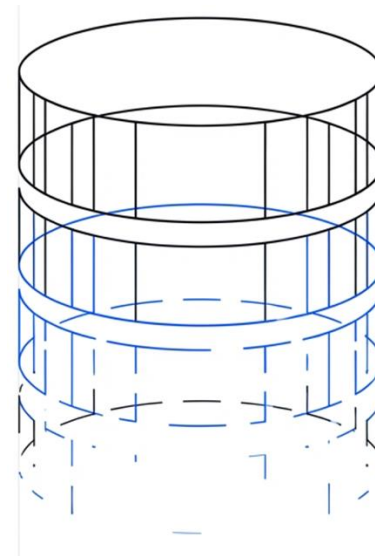


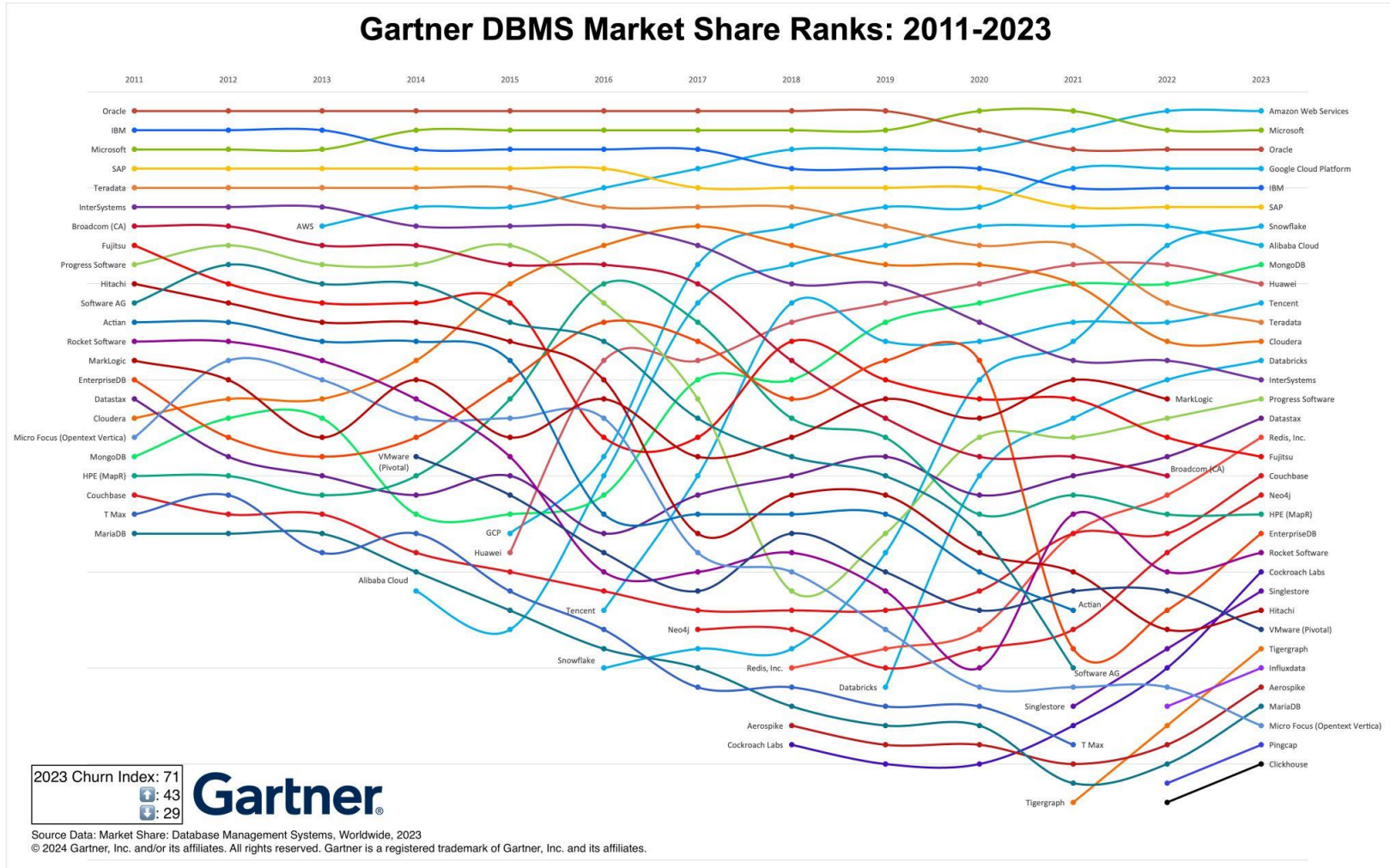
Invisible DBMS

박스(Box)에서 벗어나 API가 된 데이터베이스

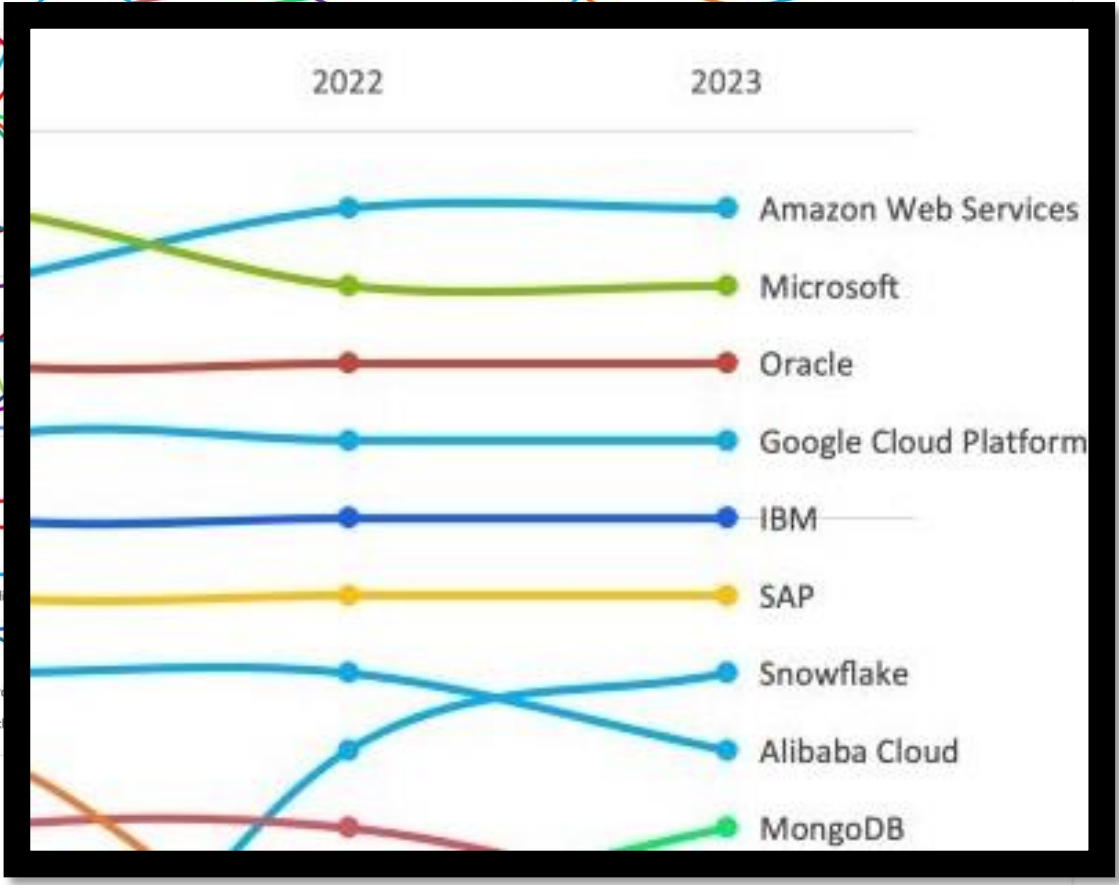
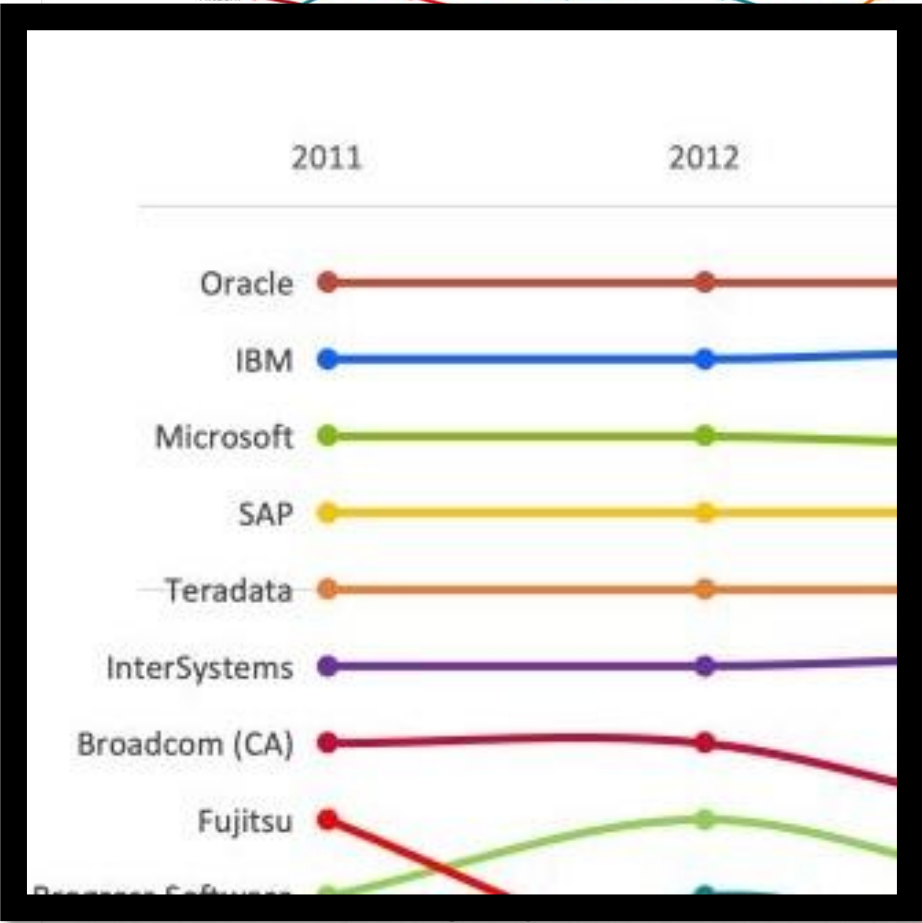
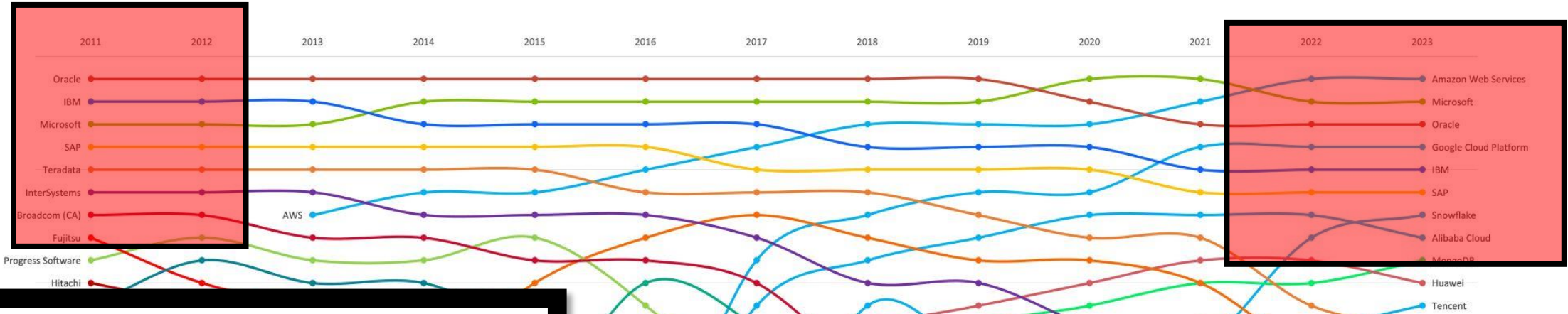
김재명



Q. 전 세계 매출 1위 DBMS 기업은?



Gartner DBMS Market Share Ranks: 2011-2023



Agenda

- Invisible DBMS
- Box 시대의 데이터베이스
- Amazon Aurora의 설계 철학
- Distributed SQL
- + Why PostgreSQL in 2026?

Inivisible Database

- 기술이 고도화 됨에 따라 데이터베이스의 존재감이 흐려짐
- 인프라에서 API로의 진화



지금은 보이지 않는 존재



과거에는 물리적인 실체

Jae M. Kim

Conclusion: Towards the 'Invisible' Database

The last five years have been a journey of databases evolving from “subjects of manual management” to “autonomous intelligent partners.” If 2021’s tuning automation started by lightening the engineer’s load, 2025’s agent and branching technologies have opened a paradigm where AI becomes the primary operator of data.

Interestingly, as the technology becomes more sophisticated, the database’s presence becomes fainter. Vector search was absorbed as a feature, and AI handles complex index management. Branching has melted from infrastructure tasks into developer tools. Furthermore, the *de facto* standardization of PostgreSQL implies that even the “choice” of a database is fading into the background. As Postgres compatibility becomes the universal dialect, the cognitive load of selecting an engine disappears—making the infrastructure truly invisible.

However, invisibility does not mean absence. As we hand the keys to AI agents, the database must evolve into a **secure safety net** that enforces the guardrails application logic can no longer guarantee and handles unprecedented usage patterns from new workloads.

Ultimately, I believe the database of the future will become “**Invisible yet Indispensable**” infrastructure—automating the complex, securing the vulnerable, and allowing us to focus solely on value creation. Looking beyond 2026, I wonder what other technologies will become “taken for granted” and fade into the background. One

[1] Source: [The AI Wave Reshaping Databases](#)

Invisible DBMS를 완성하는 4가지 조건

- Zero Provisioning
- Transparent Auto-scaling
- Self-healing & Autonomous)
- Pay-as-you-go

[2] AWS Serverless Computing Principles 등 업계 표준 서버리스 데이터베이스 아키텍처 정의 참조



Box 시대의 데이터베이스

물리적 한계에 갇혔던 인프라

프로비저닝 & 수동 확장의 늪

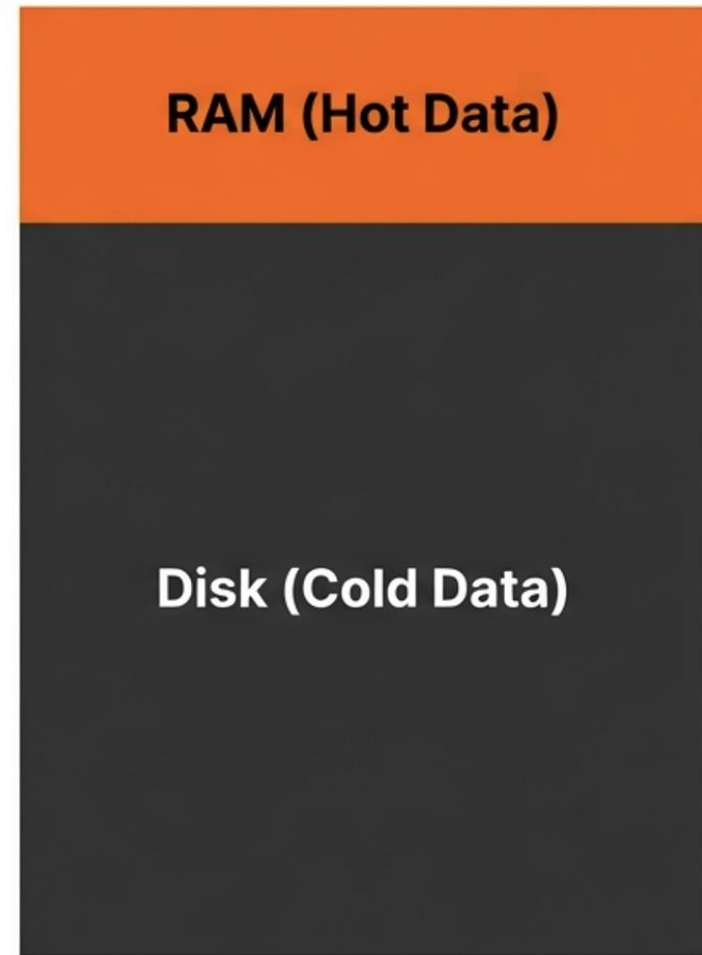
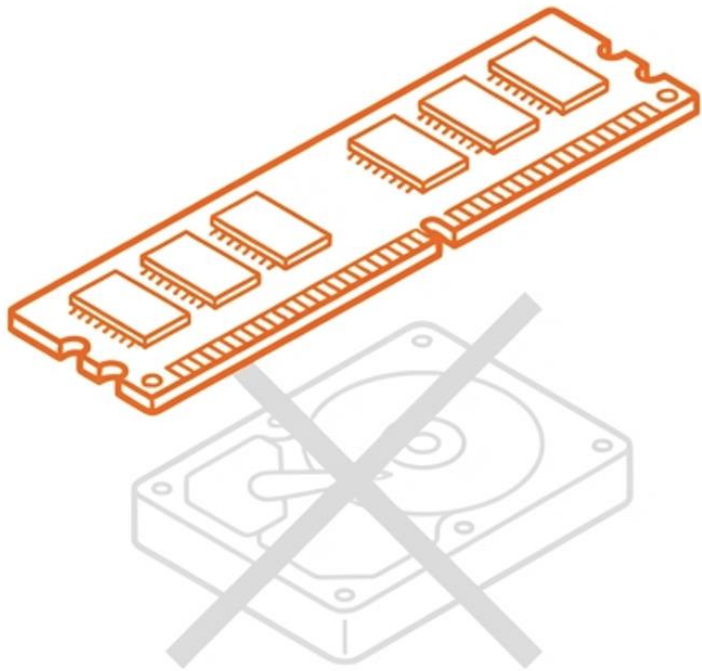
- Provisioning: 하드웨어 발주→배송(수주) → 랙 마운트/OS 설치/네트워크·스토리지 설정 → DB 설치
- Manual Scaling: 트래픽 폭주 = **스케일업(더 큰 박스)**이 유일 → 다운타임 + 밤샘 마이그레이션이 기본
- Admin 부담: 백업·패치·복구가 DBA 수작업/스크립트 의존 → 전원/디스크 장애 = 서비스 즉시 멈춤

폭발하는 데이터와 스토리지의 한계

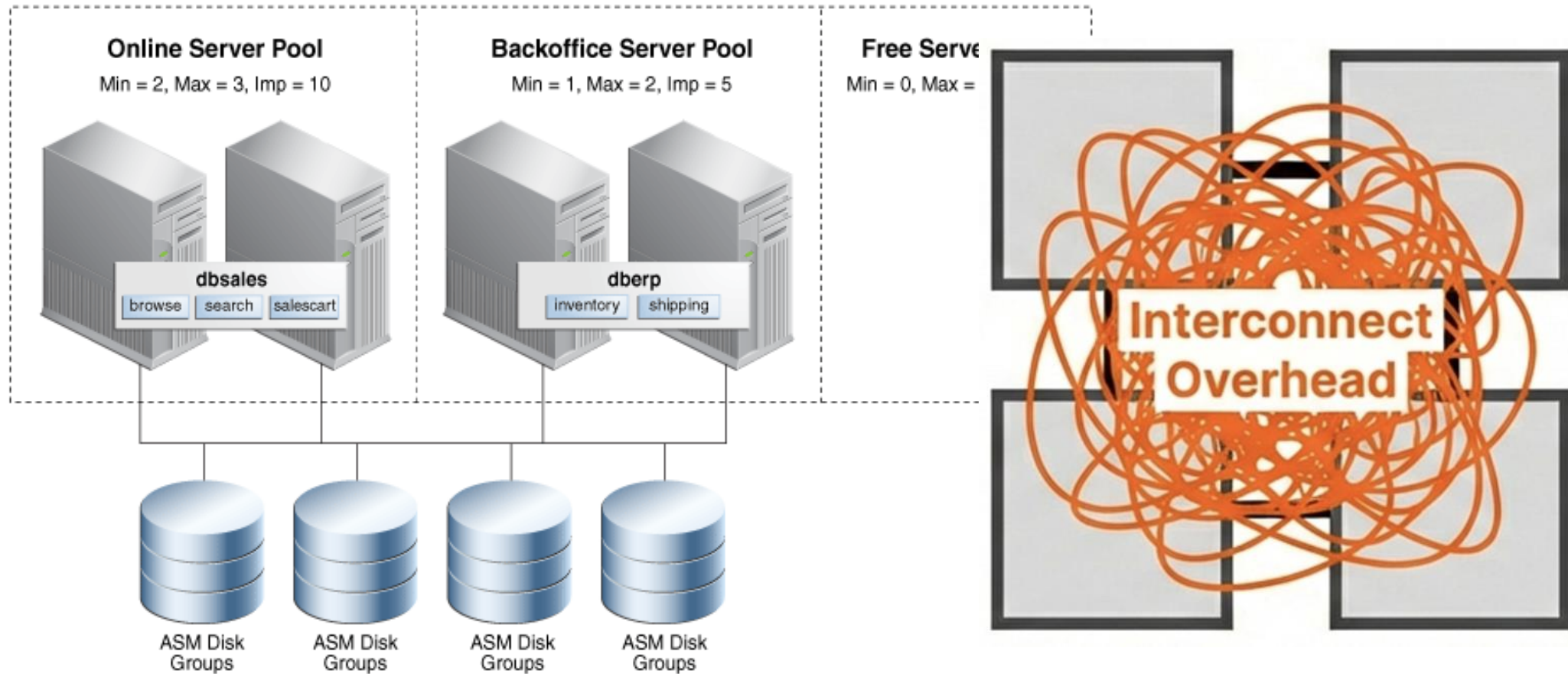


보수적인 DB시장을 뚫은 돌연변이

In-Memory DBMS (Altibase)



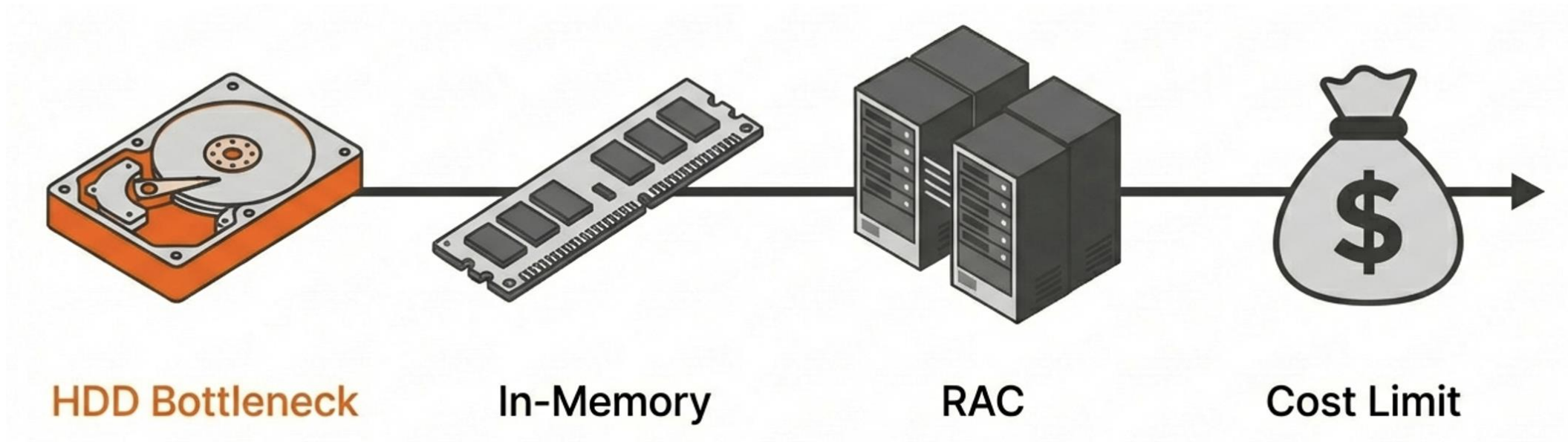
Oracle RAC



이미지 출처: <https://docs.oracle.com/en/database/oracle/oracle-database/19/racad/introduction-to-oracle-rac.html>

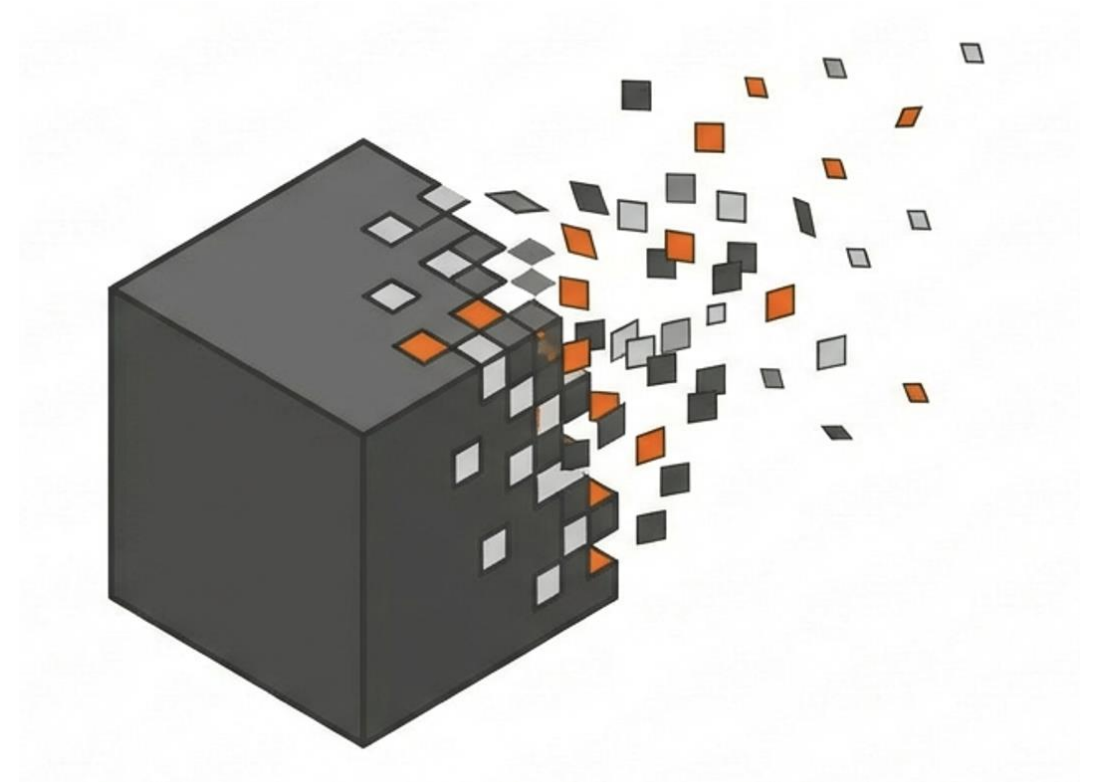
물리적 한계를 돈과 기술로 막던 시기

- 성능을 쥐어짜고, 서버를 묶으며 싸우던 시기



클라우드 시대

새로운 규모의 경제



클라우드가 바꾼 인프라

- API가 대체한 물리적 노동: 소프트웨어 정의 인프라(SDI)
- 하드디스크 병목을 깬 구원투수: 엔터프라이즈 플래시 메모리(SSD)의 부상
- 혁신의 핵심: 컴퓨팅(Compute)과 스토리지(Storage)의 분리
- DBA를 해방시킨 보이지 않는 데이터베이스 (DBaaS)

Amazon Aurora의 설계 철학

"로그가 곧 데이터베이스다"

methodologies and rapid deployments in our storage service.

3. THE LOG IS THE DATABASE

In this section, we explain why using a traditional database on a segmented replicated storage system as described in Section 2 imposes an untenable performance burden in terms of network I/Os and synchronous stalls. We then explain our approach where

Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases

Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, Xiaofeng Bao

Amazon Web Services

ABSTRACT

Amazon Aurora is a relational database service for OLTP workloads offered as part of Amazon Web Services (AWS). In this paper, we describe the architecture of Aurora and the design considerations leading to that architecture. We believe the central constraint in high throughput data processing has moved from compute and storage to the network. Aurora brings a novel architecture to the relational database to address this constraint, most notably by pushing redo processing to a multi-tenant scale-out storage service, purpose-built for Aurora. We describe how doing so not only reduces network traffic, but also allows for fast crash recovery, failovers to replicas without loss of data, and fault-tolerant, self-healing storage. We then describe how Aurora achieves consensus on durable state across numerous storage nodes using an efficient asynchronous scheme, avoiding expensive and chatty recovery protocols. Finally, having operated Aurora as a production service for over 18 months, we share lessons we have learned from our customers on what modern cloud applications expect from their database tier.

Keywords

Databases; Distributed Systems; Log Processing; Quorum Models; Replication; Recovery; Performance; OLTP

1. INTRODUCTION

IT workloads are increasingly moving to public cloud providers. Significant reasons for this industry-wide transition include the ability to provision capacity on a flexible on-demand basis and to pay for this capacity using an operational expense as opposed to capital expense model. Many IT workloads require a relational OLTP database, providing equivalent or superior capabilities to on-premise databases is critical to support this secular transition.

In modern distributed cloud services, resilience and scalability are increasingly achieved by decoupling compute from storage [10][24][36][38][39] and by replicating storage across multiple nodes. Doing so lets us handle operations such as replacing misbehaving or unreachable hosts, adding replicas, failing over from a writer to a replica, scaling the size of a database instance up or down, etc.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions.acm.org.
SIGMOD 17, May 14–19, 2017, Chicago, IL, USA.
Copyright is held by the owner/authors. Publication rights licensed to ACM.
ACM 978-1-4503-4197-4/17/05...\$15.00
DOI: <http://dx.doi.org/10.1145/303918.3036101>

The I/O bottleneck faced by traditional database systems changes in this environment. Since I/Os can be spread across many nodes and many disks in a multi-tenant fleet, the individual disks and nodes are no longer hot. Instead, the bottleneck moves to the network between the database tier requesting I/Os and the storage tier that performs these I/Os. Beyond the basic bottlenecks of packets per second (PPS) and bandwidth, there is amplification of traffic since a performant database will issue writes out to the storage fleet in parallel. The performance of the outlier storage node, disk or network path can dominate response time.

Although most operations in a database can overlap with each other, there are several situations that require synchronous operations. These result in stalls and context switches. One such situation is a disk read due to a miss in the database buffer cache. A reading thread cannot continue until its read completes. A cache miss may also incur the extra penalty of evicting and flushing a dirty cache page to accommodate the new page. Background processing such as checkpointing and dirty page writing can reduce the occurrence of this penalty, but can also cause stalls, context switches and resource contention.

Transaction commits are another source of interference; a stall in committing one transaction can inhibit others from progressing. Handling commits with multi-phase synchronization protocols such as 2-phase commit (2PC) [3][4][5] is challenging in a cloud-scale distributed system. These protocols are intolerant of failure and high-scale distributed systems have a continual “background noise” of hard and soft failures. They are also high latency, as high scale systems are distributed across multiple data centers.



Figure 1: Move logging and storage off the database engine

Decoupling Compute and Storage

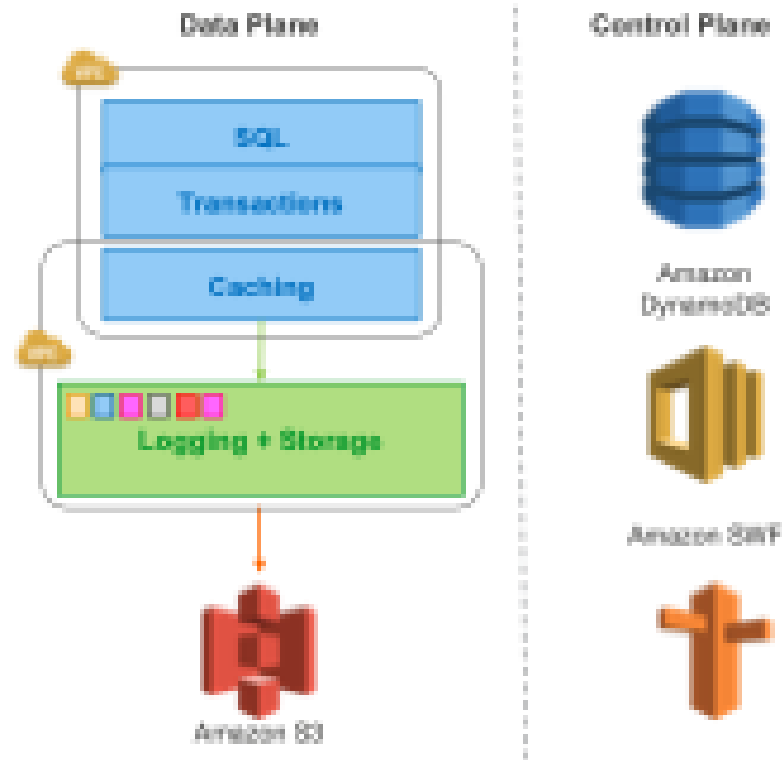


Figure 1: Move logging and storage off the database engine

네트워크 I/O의 최소화

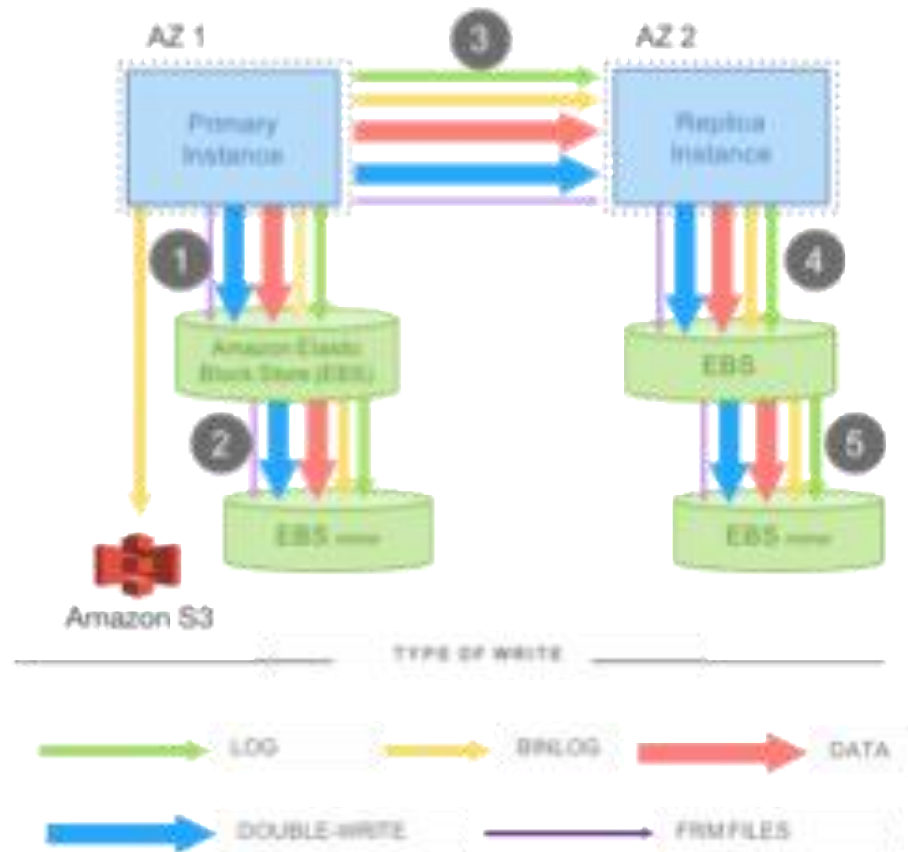


Figure 2: Network IO in mirrored MySQL

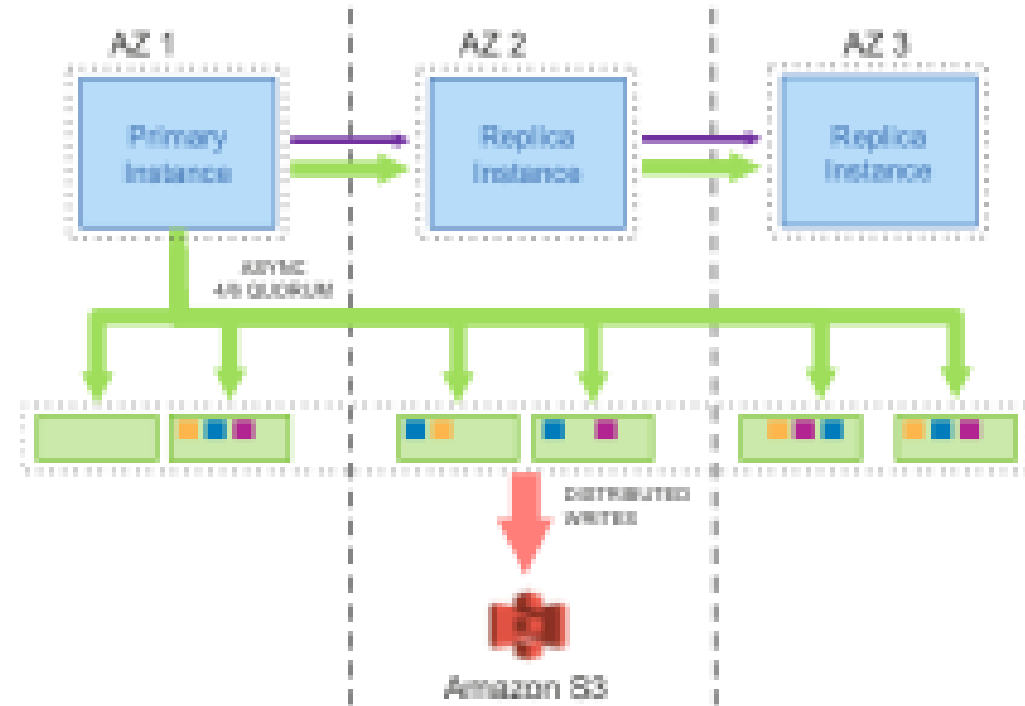


Figure 3: Network IO in Amazon Aurora

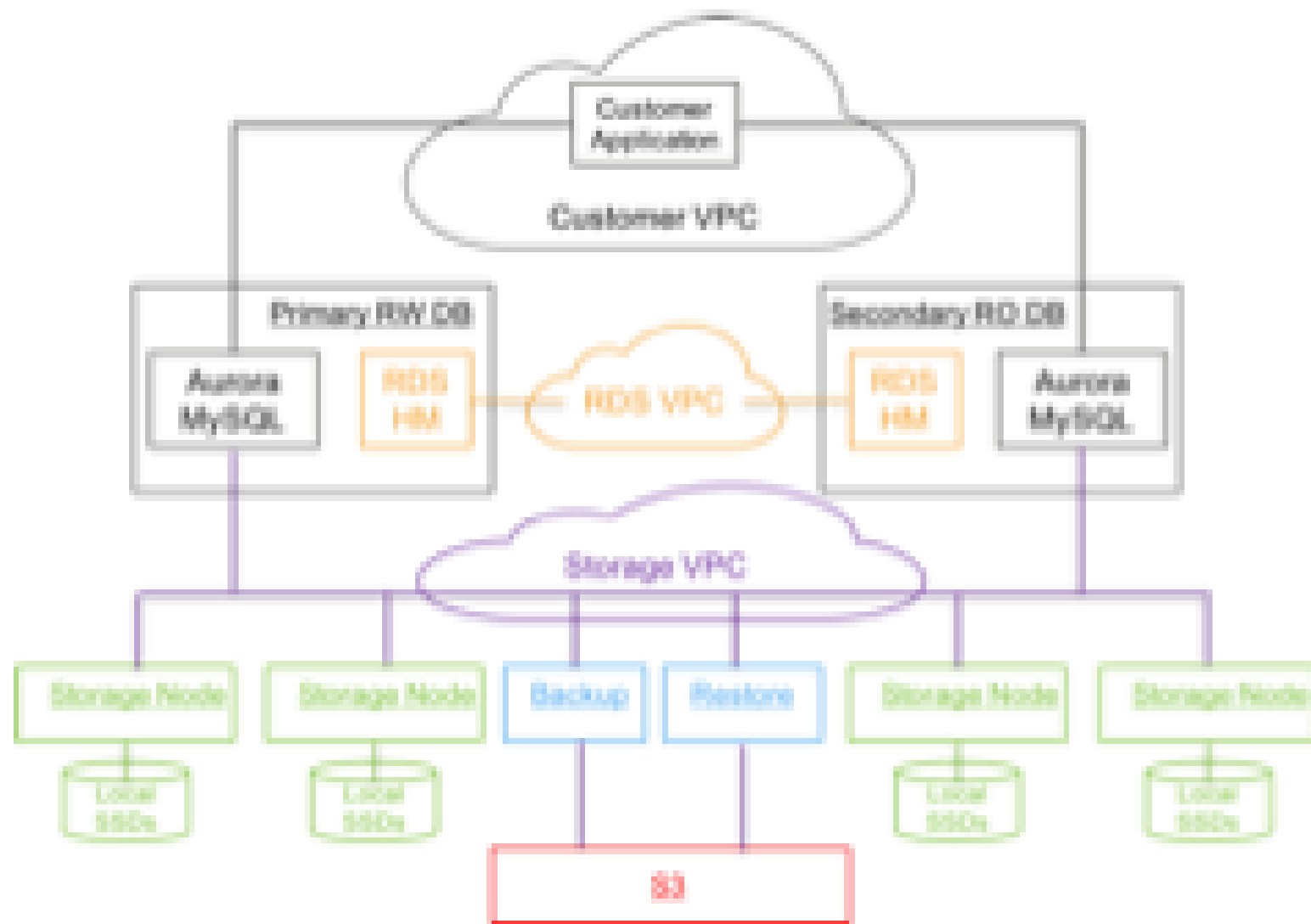


Figure 5: Aurora Architecture: A Bird's Eye View

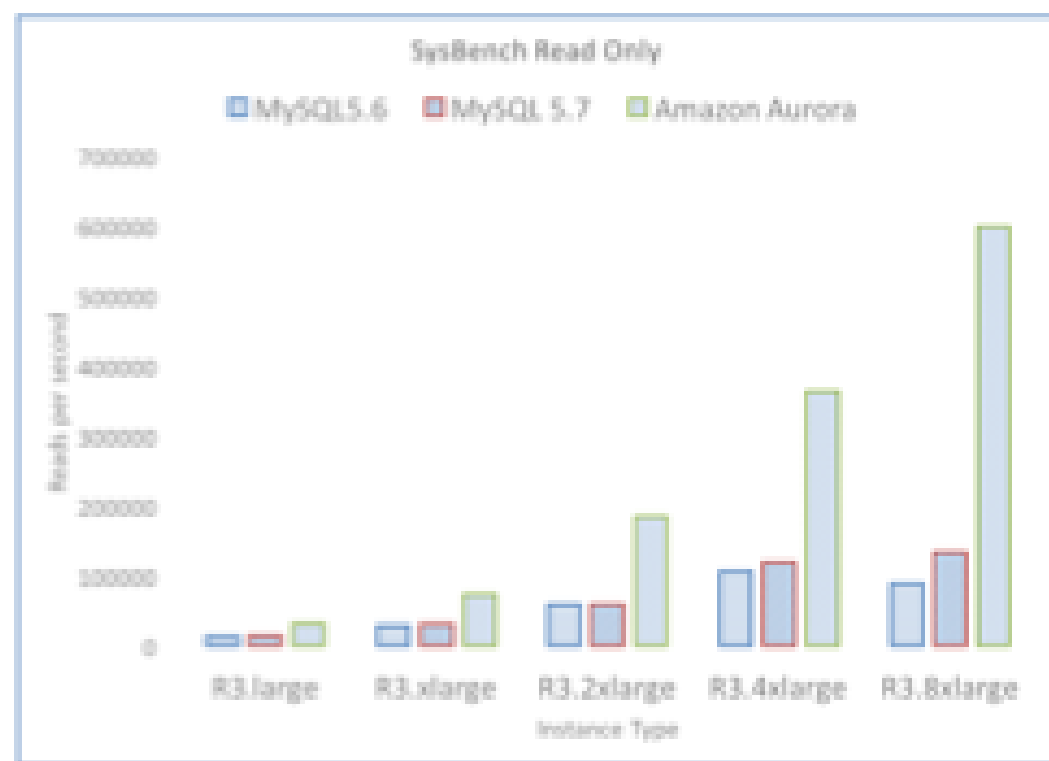


Figure 6: Aurora scales linearly for read-only workload

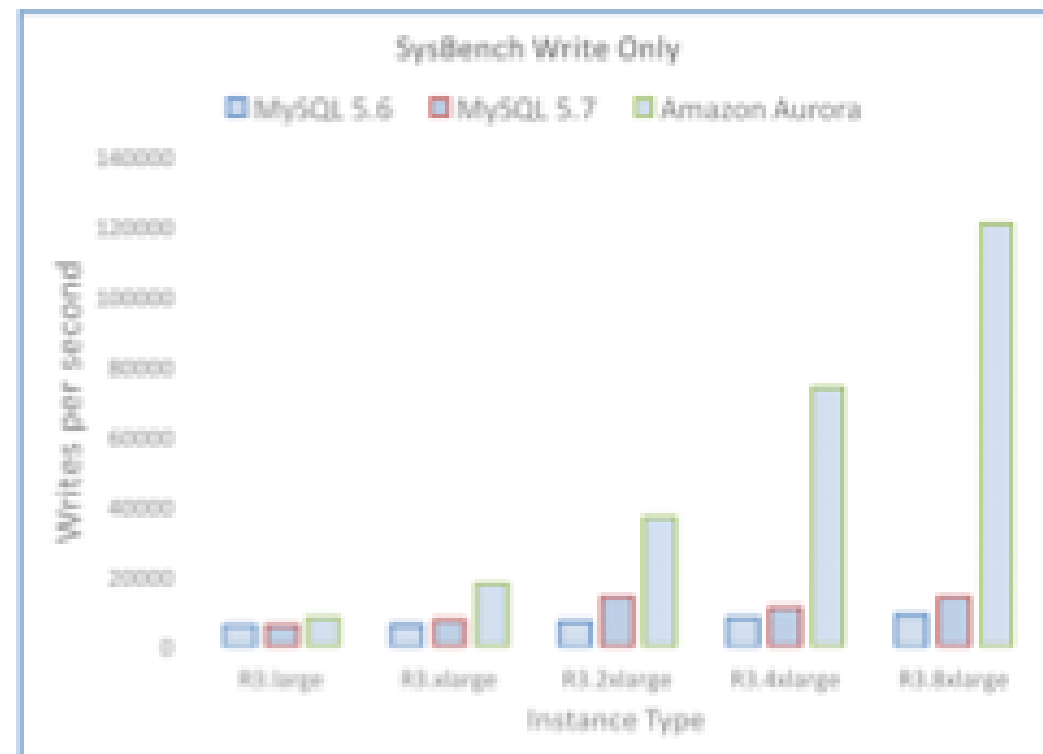


Figure 7: Aurora scales linearly for write-only workload

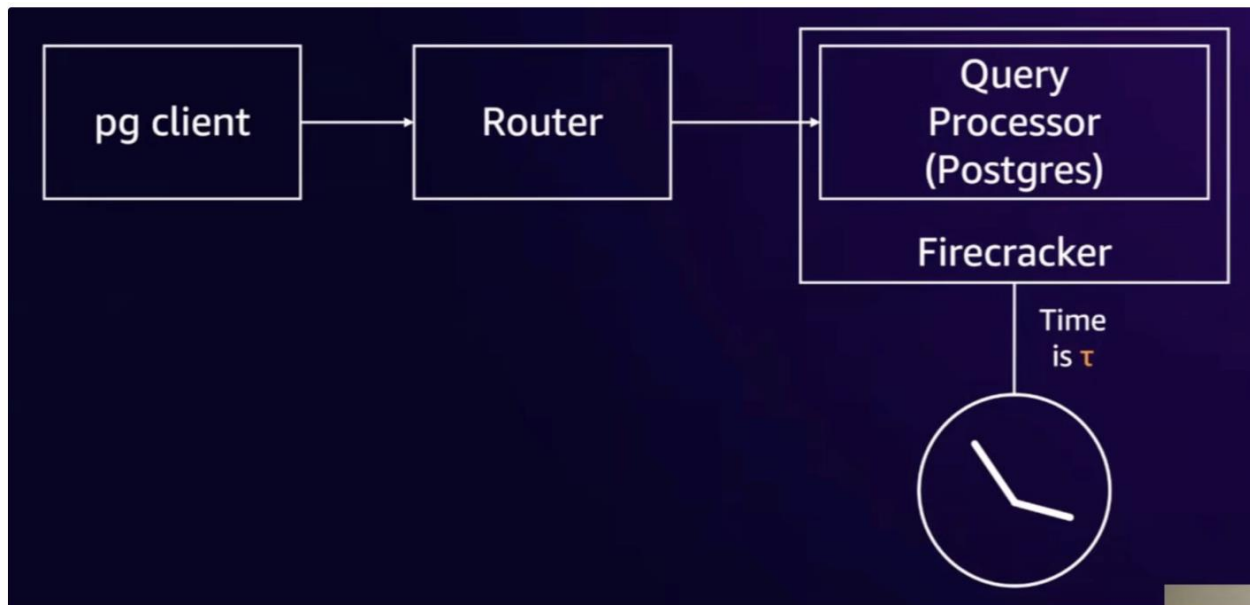
Aurora Distributed SQL (DSQL)

Next Aurora?

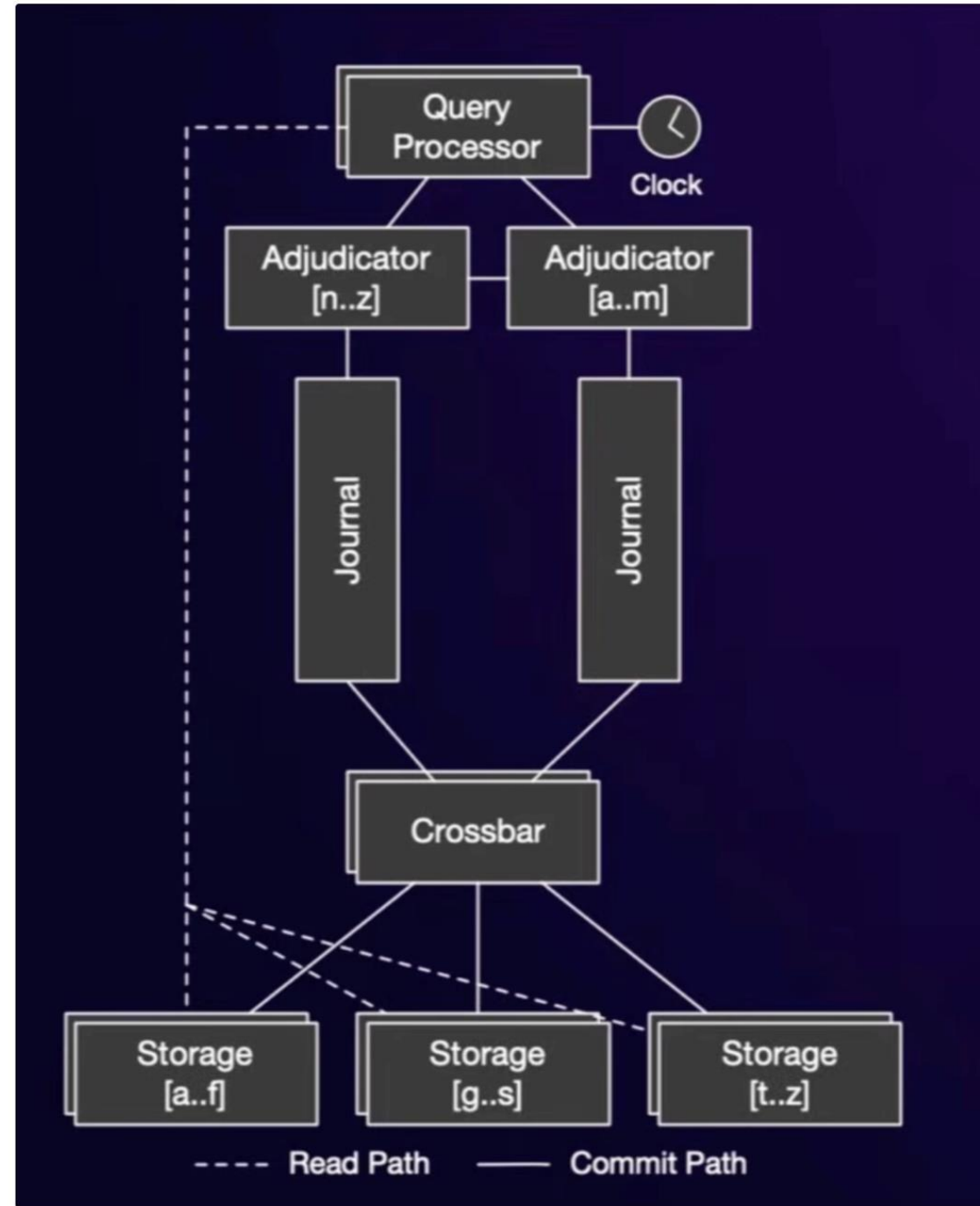
왜 다시 Serverless 데이터베이스?

- 기존 관리형 DB의 한계: 운영 자동화는 이뤘지만, 여전히 '물리적 인스턴스' 개념이 존재하고 특정 가용 영역(AZ)에 종속됨.
- DSQL의 철학 (System of first resort): 인스턴스 크기 산정이나 리전 가용성 고민 없이 즉시 연결해서 사용하는 데이터베이스.
- 극대화된 확장성: 1 QPS부터 수백만 QPS까지 대기 시간 없이 즉각적으로 확장되는 분산 시스템.
- 진정한 'API로서의 데이터베이스': 단순한 관리 편의성을 넘어선 완전한 서버리스(True Serverless) 아키텍처의 구현.

Aurora DSQL 아키텍처



이미지 출처: <https://www.youtube.com/watch?v=mK2h9VPRgnI>



이미지 출처: <https://www.youtube.com/watch?v=mK2h9VPRgnI>

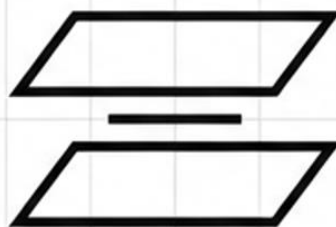
DSQL, 기존 DB의 대체재? 새로운 기회?

- 현재의 기능적 제약: microVM 및 분산 엔진 아키텍처 특성상, 기존 Aurora PostgreSQL 대비 일부 기능 지원이 제한적임.
- 완전한 대체가 아닌 새로운 패러다임: 기존 DB의 단순 교체보다는, DSQL만의 강점을 살릴 수 있는 완전히 새로운 워크로드(Killer App) 탐구 단계.
- 명확한 기술적 트레이드오프:
 - ▼ 구조적 한계: 분산 처리로 인한 개별 쿼리 지연, 초대형 복잡 쿼리 처리 제약, 쓰기 경합 시 OCC(낙관적 동시성 제어) 충돌 비용 발생.
 - 🚀 독보적 강점: 압도적인 확장성과 '진정한 서버리스' 아키텍처 구현, 타 DB 대비 월등한 비용 효율성.
- 최적의 타겟 워크로드: 위 제약 사항들이 치명적이지 않은 현대적인 마이크로서비스 및 AI 애플리케이션에 가장 혁신적인 선택지.

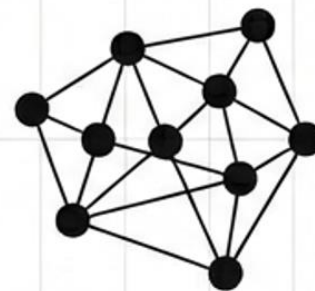
중간정리



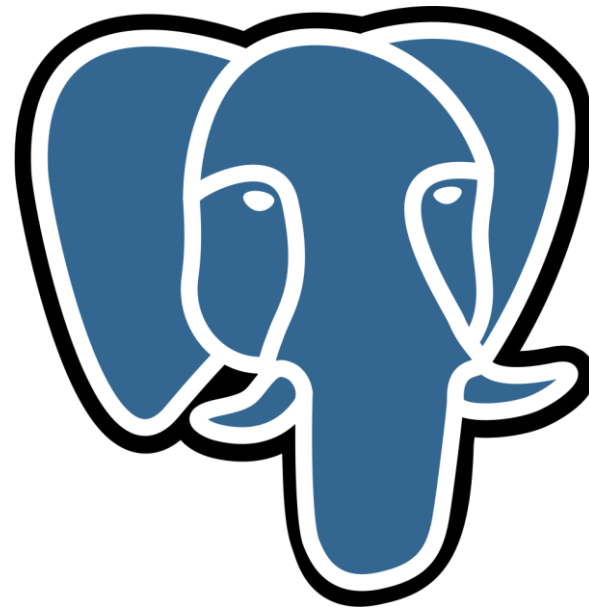
Box Era
(Physical)



Cloud Era
(Decoupled)



Serverless Era
(Distributed)



Why PostgreSQL in 2026?

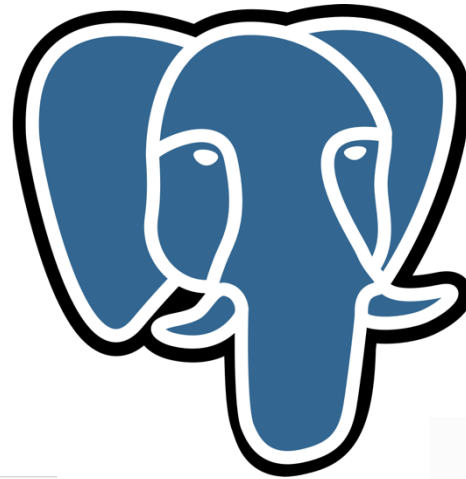
왜 당신은 PostgreSQL을 자주 접하고 있는가?



MyISAM
=> InnoDB



Amazon Aurora
PostgreSQL



AlloyDB



amazon
REDSHIFT



이미지 출처:

<https://www.snowflake.com/en/blog/snowflake-postgres-enterprise-ai-database/>

이미지 출처: <https://hostnoc.com/databricks-to-acquire-neon>

감사합니다.